Progress Report

Day 6 — JWT Authentication Implementation
Task Management MERN Stack Platform
Shafin Nigamana | 24 May 2026

1. Day-6 Objective

The goal for Day 6 was to implement JWT-based authentication, secure user passwords using bcrypt hashing, and protect backend routes using authentication middleware.

2. Tasks Completed

User Registration

- Implemented POST /api/auth/register endpoint
- Added user creation flow using MongoDB
- Configured bcrypt password hashing before database storage
- Generated JWT token after successful registration

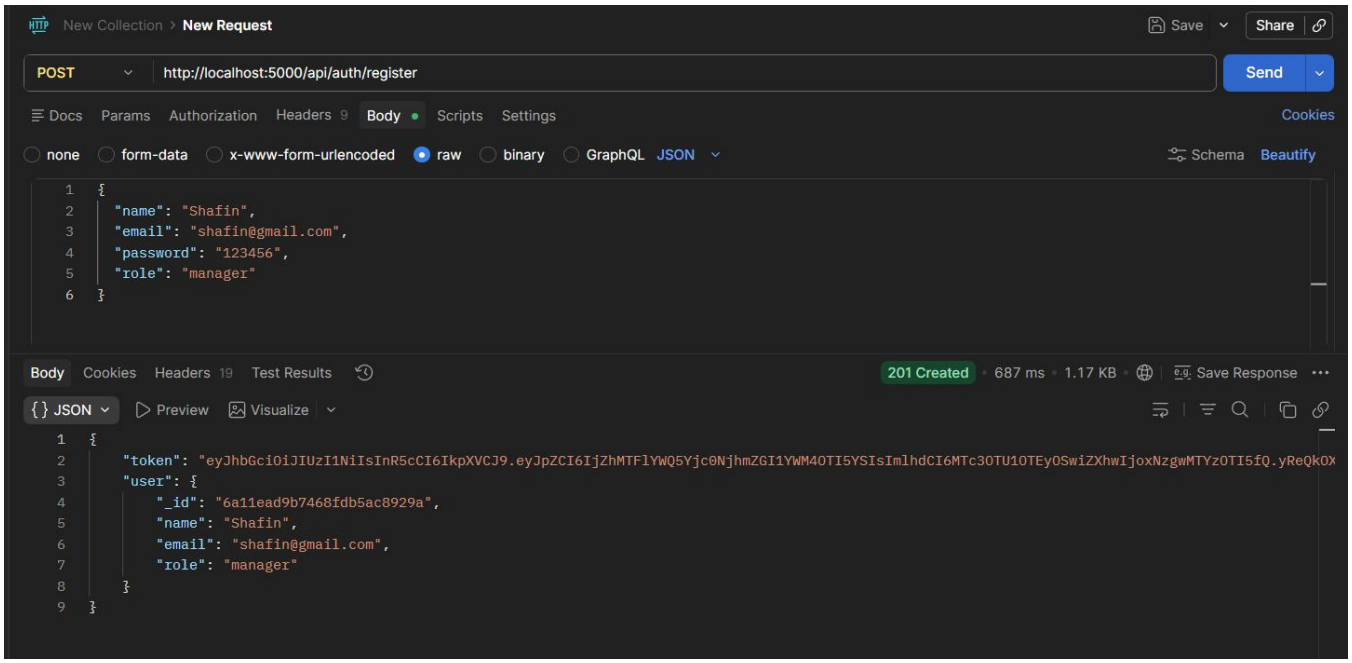

Figure 2.1 — User registration endpoint tested successfully.

User Login

- Implemented POST /api/auth/login endpoint
- Verified user credentials using bcrypt password comparison
- Generated JWT token after successful login
- Returned authenticated user information without passwordHash

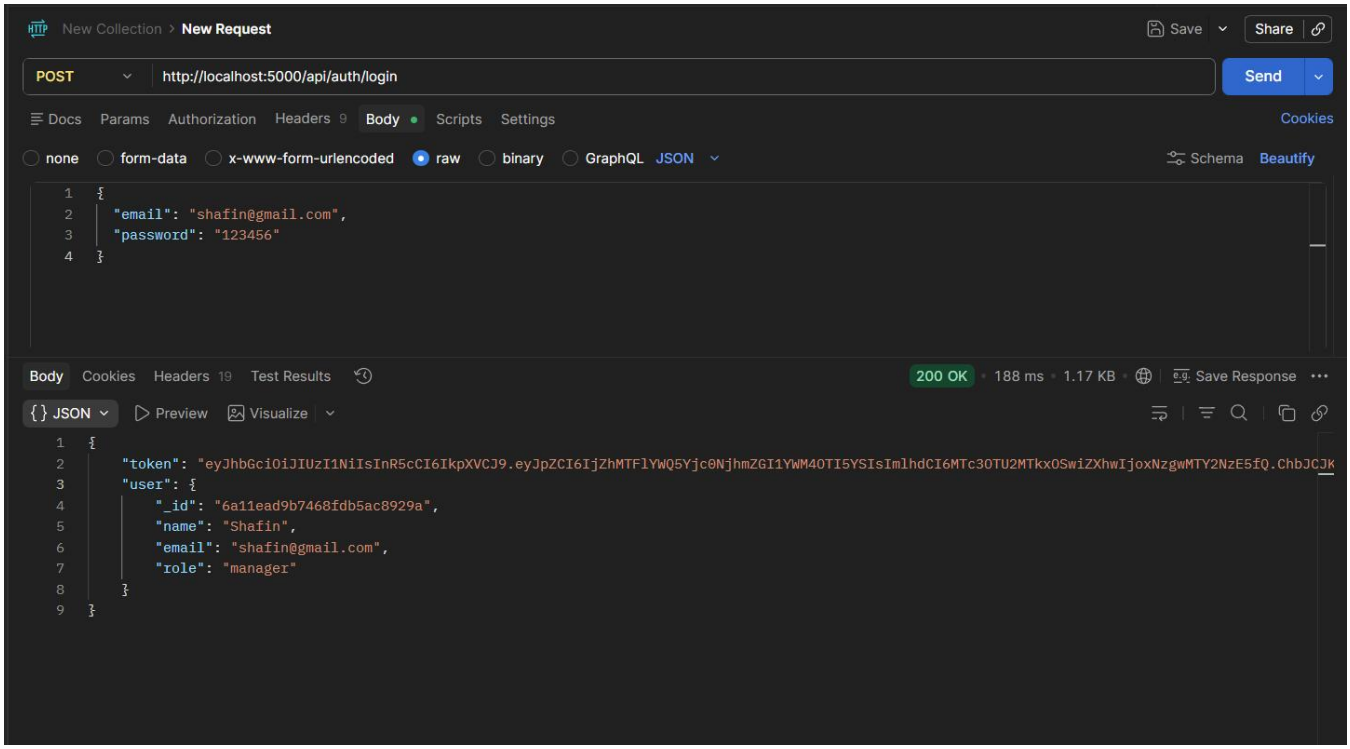

Figure 2.2 — User login endpoint returning JWT token.

JWT Authentication Middleware

- Implemented authentication middleware using jsonwebtoken
- Configured Bearer token extraction from request headers
- Added JWT verification using jwt.verify()
- Attached authenticated user information to req.user

Figure 2.3 — JWT authentication middleware integrated into backend routes.

Protected Route Implementation

- Implemented protected GET /api/auth/me route
- Verified authenticated user access using Bearer token
- Restricted unauthenticated requests using middleware protection

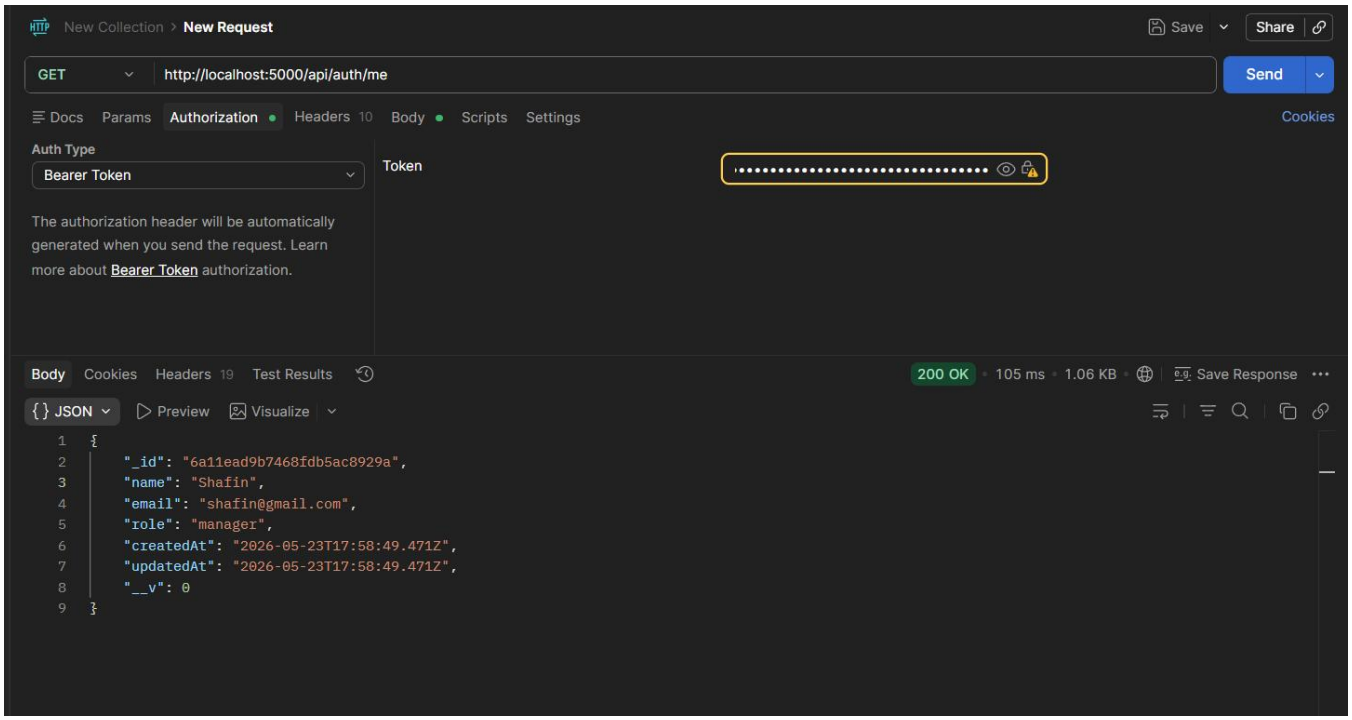

Figure 2.4 — Protected /me route returning authenticated user data.

Unauthorized Access Verification

- Tested protected route access without JWT token
- Verified backend returns unauthorized response correctly

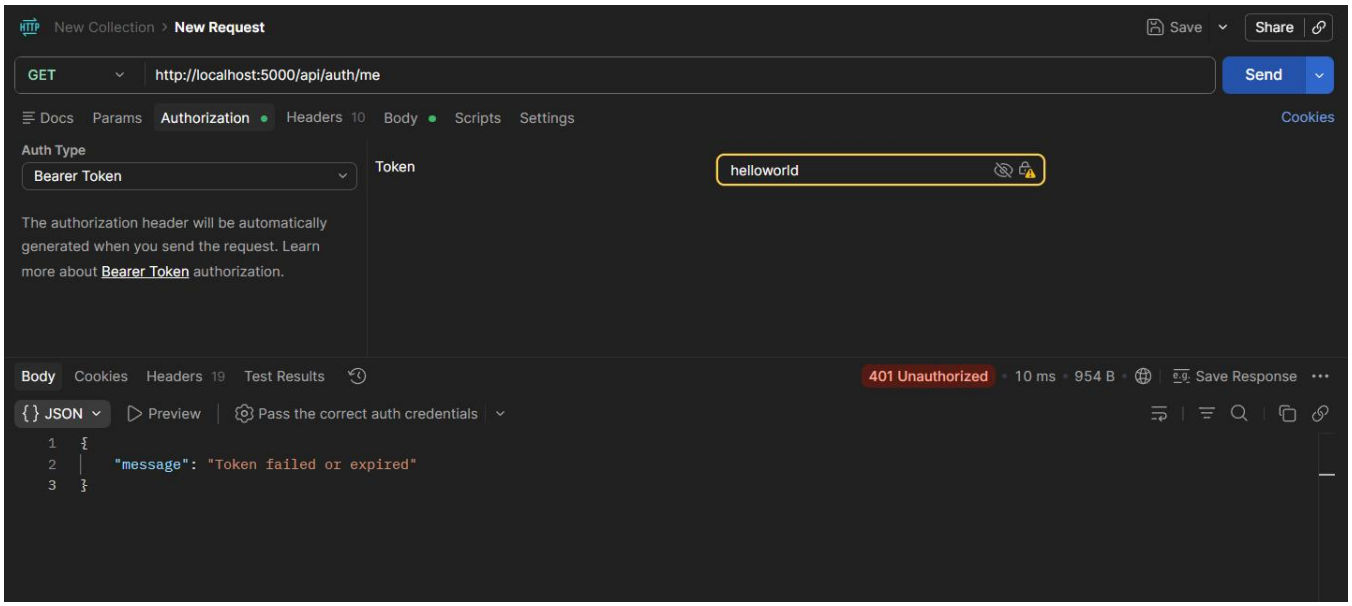

Figure 2.5 — Unauthorized request blocked by authentication middleware.

Password Hash Verification

- Verified passwords are stored as bcrypt hashes inside MongoDB Atlas
- Confirmed plain text passwords are not stored in the database

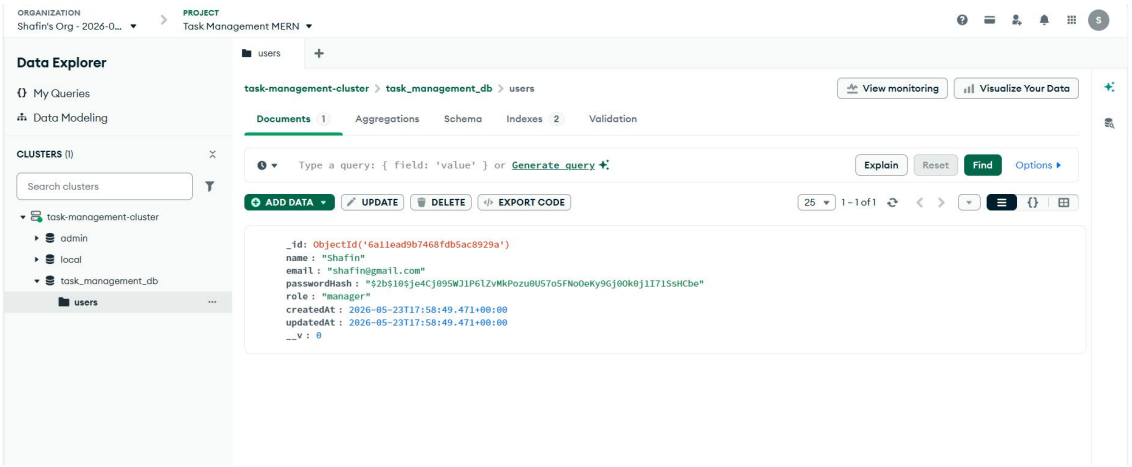

Figure 2.6 — Password stored as bcrypt hash inside MongoDB collection.

Repository Update

- Committed authentication implementation changes using Git workflow
- Pushed latest backend authentication updates to GitHub repository

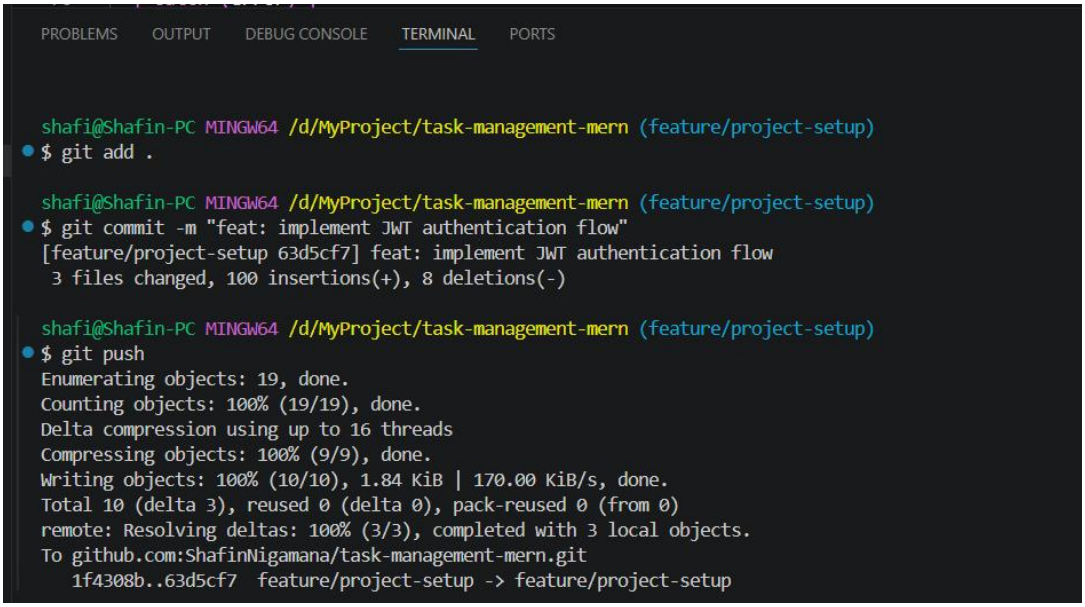

Figure 2.7 — Day 6 JWT authentication implementation pushed to GitHub.

3. Current Authentication Flow

Current authentication flow:

Register User

↓

Hash Password

↓

Store User

↓

Generate JWT

↓

Login User

↓

Verify Token

↓

Access Protected Routes

4. Challenges Faced

- Ensuring passwordHash is stored securely instead of plain text passwords
- Configuring middleware to correctly extract and verify Bearer tokens from request headers
- Protecting backend routes while keeping the implementation structure minimal and clean

5. Next Steps (Day-7)

- Begin Team CRUD endpoint implementation
- Add manager-only authorization middleware
- Prepare backend role-based access control flow
- Continue API testing using Postman

6. Conclusion

Day 6 focused on implementing backend authentication using JWT and bcrypt. User registration, login, token verification, and protected route handling were implemented successfully. Password hashing and authentication middleware were verified using Postman and MongoDB Atlas.